Radiant College, Manigram Butwal

[ITEX 103] Object Oriented Programming

B Tech Ed. – 2ND SEMESTER

Unit – 7

(Exception Handling)

Objective:

1. **Understand the idea of error and exceptions**
2. **Implement ideas to handle exceptions in C++**
3. **Implement rethrowing an exception in C++**
4. **Handle various types of exceptions in C++**

Introduction:

Exception handling in C++ provides a mechanism for handling runtime errors (e.g., division by zero, file not found) in a structured manner, ensuring program robustness and clean error management. It separates error-handling code from normal program flow.

Error Handling:

- **Definition:** Error handling manages unexpected conditions during program execution.
    - **Types of Errors in C++:**
        - **Compile-time errors: Syntax errors (e.g., missing ;).**
        - **Runtime errors: Errors during execution (e.g., accessing a null pointer, division by zero).**
        - **Logical errors: Incorrect logic (e.g., wrong calculations).**
    - **Traditional Error Handling**
        - **Uses return codes or error flags (`e.g., if (func() == -1) { /* error */ }`).**
        - **Drawbacks: Code clutter, error-prone, and hard to maintain.**

- **C++ Approach: Exception handling with try, catch, and throw provides a cleaner, type-safe mechanism.**

Exception Handling Constructs

- **Exception:** An exception is a mechanism that allows a program to handle unexpected or error conditions at runtime without crashing

- **Exception handling:** Exception handling in C++ is a mechanism to detect, report, and recover from errors during program execution in a structured and predictable way.

  Instead of allowing the program to crash when an error occurs (like dividing by zero or accessing out-of-bounds memory), exception handling provides a clean way to **"throw"** an error and **"catch"** it elsewhere in the code to manage it gracefully.

- **Core Constructs:**

  - **try**: Encloses code that might throw an exception.

```
try{

   // Code that may throw error/exception
}
```

  - **catch: Handles exceptions thrown in the try block.**

```
catch(const std:exception& e){

     // Handle exception
}
```

  - **throw: Throws an exception object.**

```
if(condition)
```

```
{
      throw std::runtime_error("Error occurred");
}
```

- **Flow:**
  - Code in **try** block executes.
  - If an exception is thrown, control jumps to the matching **catch** block.
  - Execution resumes after the try-catch block.

- **Example**

```
#include <stdexcept>
#include<iostream>
int main() {
    try {
        int x = 10, y = 0;
        if (y == 0) throw std::runtime_error("Division by zero");
        std::cout << x / y << std::endl;
    } catch (const std::runtime_error& e) {
        std::cout << "Error: " << e.what() << std::endl;
    }
    return 0;
}
```

- **Output: Error: Division by zero**

Multiple Exception Handling

- **Concept**: Multiple exception handling in C++ allows a program to handle different types of exceptions thrown within a **try** block using multiple catch blocks. Each **catch** block is designed to handle a specific exception type, ensuring robust and type-safe error management.

- **Key Concepts**

  - **Purpose:** Handle various exception types (e.g., int, std::string, std::exception, custom classes) thrown during program execution.

  - **Mechanism:** A try block encloses code that may throw exceptions, followed by multiple catch blocks, each matching a specific exception type.

  - **Type Matching:** C++ matches exceptions by their type, not value, using the first matching catch block.

  - **Order of Catch Blocks:** Specific exception types must be caught before general ones to avoid shadowing (e.g., catch std::runtime_error before std::exception).

- **Core Constructs**
  - **try:** Contains code that might throw exceptions.

```
try {
// Code that may throw error
}
```

  - **catch:** Handles a specific exception type. Use references (const T&) to avoid copying and support polymorphism.

```
catch(const exceptiontype1 e){
//catch exceptiontype1 (may be int)

} catch(const exceptiontype2 e){

//catch exceptiontype2 (may be float)
```

```
}catch (const exceptiontype3 e){
// catch exceptiontype3 may be any other exception
}
```

- **Catch-All**: catch (...) handles any uncaught exceptions.

```
catch(...){
// Handle any type of exceptions
}
```

- **Example:**

```
#include <stdexcept>
#include <string>
#include <iostream>

class MyException: public std::exception {
public:
    const char* what() const noexcept override {
        return "My Custom Exception";
    }
};

void throwMultipleException(int a) {
    switch (a) {
        case 1: throw 1; // Integer
        case 2: throw 1.1; // Double
        case 3: throw std::string("String Exception"); // String
        case 4: throw MyException(); // Custom exception
        case 5: throw std::runtime_error("Runtime Error"); //
Standard exception
        default: throw std::exception(); // Base exception
    }
}

int main() {
```

```
    try {
        throwMultipleException(4); // Test with values 1-5 or other
    } catch (const int& i) {
        std::cout << "Integer exception: " << i << std::endl;
    } catch (const double& d) {
        std::cout << "Double exception: " << d << std::endl;
    } catch (const std::string& s) {
        std::cout << "String exception: " << s << std::endl;
    } catch (const MyException& e) {
        std::cout << "Custom exception: " << e.what() << std::endl;
    } catch (const std::exception& e) {
        std::cout << "Standard exception: " << e.what() << std::endl;
    } catch (...) {
        std::cout << "Unknown exception occurred" << std::endl;
    }
    std::cout << "After exception handling" << std::endl;
    return 0;
}
```

Explanation of Example

- **Throwing:** The throwMultipleException function throws different types based on input (int, double, std::string, MyException, std::runtime_error, std::exception).
- **Catching:**
    - Each **catch** block handles a specific type in order of specificity.
    - catch (const std::exception&) **catches** std::runtime_error and MyException (if derived from std::exception).
    - catch (...) is last, handling any uncaught types (e.g., raw types like int if not caught earlier).
- **Output for throwMultipleException(4):**

```
Custom exception: My Custom Exception
After exception handling
```

Catching All Exceptions

- **Concept:** A catch-all block **(catch (...))** handles any exception not caught by specific **catch** blocks.

- **Syntax**

```
catch( ... ) {
   cout << “Unknown exception caught” <<std::endl;
}
```

- **Use Case:**
  - Fallback for unexpected exceptions
  - Useful for logging or cleanup

Example:

```
#include <iostream>
#include <stdexcept>
int main() {
    try {
        throw 42; // Throws an int (not std::exception)
    } catch (const std::runtime_error& e) {
        std::cout << "Runtime error: " << e.what() <<
std::endl;
    } catch (...) {
        std::cout << "Caught an unknown exception" <<
std::endl;
    }
    return 0;
}
```

- **Output:** Caught an unknown exception

- **Caution:**
  - Cannot access exception details in catch(...)
  - Use sparingly to avoid masking errors.

Exception with Arguments

- **Concept:** Exceptions can carry data to provide detailed error information.
- **Standard Exceptions:** Use std::runtime_error or similar with a message string.
- **Custom Exceptions:**
  - Derive from std::exception and override what().
  - Add fields for additional data (e.g., error code).

Example:

```
#include <iostream>
#include <string>
#include <exception>

class MyException : public std::exception {
    std::string message;
    int code;

public:
    MyException(const std::string& msg, int c) {
        message = msg;
        code = c;
    }

    const char* what() const noexcept override {
        return message.c_str();
    }

    int getCode() const {
        return code;
    }
};

int main() {
    try {
```

```
        throw MyException("File not found", 404);
    } catch (const MyException& e) {
        std::cout << "Error: " << e.what()
                  << ", Code: " << e.getCode() <<
std::endl;
    }

    return 0;
}
```

Output: Error: File not found, Code: 404

Key Points:

- **Use** noexcept **for** what() **to comply with** std::exception.
- Custom exceptions are useful for domain-specific errors.

Re-throwing an Exception

- **Concept:** A caught exception can be re-thrown to propagate it to an outer try-catch block

- **Syntax:**

```
catch( const exception& e){

    // partial handling
     Throw; // Re-throw same exception
}
```

Example:

```
#include <iostream>
#include <stdexcept>
int main() {
    try {
        try {
            throw std::runtime_error("Inner error");
        } catch (const std::runtime_error& e) {
            std::cout << "Inner catch: " << e.what() <<
std::endl;
            throw; // Re-throw
        }
    } catch (const std::runtime_error& e) {
        std::cout << "Outer catch: " << e.what() << std::endl;
    }
    return 0;
}
```

Output:

Inner catch: Inner error

Outer catch: Inner error

Key Points:

- Use throw (no arguments) to preserve the original exception
- Avoid modifying the exception unless necessary

Handling Uncaught and Unexpected Exceptions

Handling Uncaught and Unexpected Exceptions in C++

Handling uncaught and unexpected exceptions in C++ requires careful use of try-catch blocks, exception handlers, and cleanup mechanisms to ensure robust and safe program behavior. Below are key strategies and best practices tailored for C++.

Key Approaches

Global Exception Handling

Use a top-level try-catch block in main() or critical functions to catch unhandled exceptions.

```
#include <iostream>
#include <exception>
#include <fstream>

void logError(const std::exception& e) {
    std::ofstream log("error.log", std::ios::app);
    log << "Error: " << e.what() << std::endl;
}

int main() {
    try {
        // Main application code
        throw std::runtime_error("Unexpected error occurred");
    } catch (const std::exception& e) {
        logError(e);
        std::cerr << "Caught exception: " << e.what() <<
std::endl;
        return 1;
    } catch (...) {
        std::ofstream log("error.log", std::ios::app);
```

```
        log << "Unknown exception occurred" << std::endl;
        std::cerr << "Unknown error occurred" << std::endl;
        return 1;
    }
    return 0;
}
```

Std::set_terminate

Customize the behavior for uncaught exceptions by setting a terminate handler.

```
#include <iostream>
#include <exception>
#include <fstream>
#include <cstdlib>

void customTerminate() {
    std::cerr << "Uncaught exception. Terminating program." << std::endl;
    std::ofstream log("error.log", std::ios::app);
    log << "Program terminated due to uncaught exception" << std::endl;
    std::abort();
}

int main() {
    std::set_terminate(customTerminate);
    throw std::runtime_error("This will trigger terminate");
    return 0;
}
```